

**PRAKTIKUM BASIS DATA**

**MODUL 6**

**(06\_Scripting\_Subquery)**



Oleh:

(Naufal Kafabih Khalwani) (103122400036)

**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK**

**DIREKTORAT KAMPUS PURWOKERTO**

**UNIVERSITAS TELKOM**

**2026**

## 1. Definisi Subquery dan tipe-tipe subquery

Subquery adalah suatu perintah query yang berada di dalam query lain dan digunakan untuk membantu menghasilkan data yang dibutuhkan oleh query utama. Subquery biasanya ditempatkan di dalam klausa seperti SELECT, FROM, atau WHERE. Dengan menggunakan subquery, proses pengambilan data menjadi lebih fleksibel karena memungkinkan pengambilan data berdasarkan hasil dari query lain.

Terdapat beberapa tipe subquery. Pertama adalah **single row subquery**, yaitu subquery yang menghasilkan satu baris data dan biasanya digunakan dengan operator seperti tanda sama dengan (=). Kedua adalah **multiple row subquery**, yaitu subquery yang menghasilkan lebih dari satu baris data dan digunakan dengan operator seperti IN, ANY, atau ALL. Ketiga adalah **multiple column subquery**, yaitu subquery yang menghasilkan lebih dari satu kolom sekaligus. Terakhir adalah **correlated subquery**, yaitu subquery yang bergantung pada query utama karena dijalankan berulang untuk setiap baris pada query luar.

## 2. Hal-hal yang harus diperhatikan dalam subquery

Dalam menggunakan subquery, terdapat beberapa hal penting yang harus diperhatikan. Pertama, hasil dari subquery harus sesuai dengan operator yang digunakan, misalnya jika menggunakan tanda sama dengan (=), maka subquery harus menghasilkan satu nilai saja. Kedua, pemilihan operator harus tepat, seperti menggunakan IN untuk banyak data atau = untuk satu data. Ketiga, tipe data antara hasil subquery dan query utama harus sesuai agar tidak terjadi error. Keempat, penggunaan subquery perlu memperhatikan performa karena subquery yang kompleks dapat memperlambat eksekusi query. Kelima, penggunaan alias sangat penting terutama pada correlated subquery agar tidak terjadi kebingungan antara query dalam dan luar. Selain itu, sebaiknya hindari penggunaan subquery yang terlalu dalam atau berlapis-lapis karena akan membuat query sulit dipahami dan tidak efisien.

## 3. Query DDL dan DML Bioskop

Untuk membuat struktur database bioskop, digunakan perintah DDL (Data Definition Language) seperti CREATE TABLE untuk membuat tabel film, penonton, jadwal, dan tiket. Setiap tabel harus diberi nama sesuai format yang diminta, yaitu ditambahkan dengan NIM, misalnya film\_123456, penonton\_123456, dan seterusnya. Setelah tabel dibuat, digunakan perintah DML (Data Manipulation Language) seperti INSERT INTO untuk memasukkan data ke dalam tabel tersebut. Data yang dimasukkan meliputi informasi film seperti judul dan durasi, data penonton seperti nama dan umur, serta data jadwal dan tiket yang menghubungkan antara film dan penonton. Dengan demikian, database dapat digunakan untuk menjalankan query selanjutnya.

#### **4. Subquery untuk menampilkan film dengan durasi terpendek dan umur termuda**

Untuk menampilkan judul film dengan durasi terpendek dan penonton dengan umur termuda, digunakan subquery yang mencari nilai minimum dari masing-masing data. Subquery pertama digunakan untuk mendapatkan durasi terkecil dari tabel film, sedangkan subquery kedua digunakan untuk mendapatkan umur terkecil dari tabel penonton. Kedua hasil tersebut kemudian digunakan dalam query utama untuk memfilter data yang sesuai. Dengan bantuan join antar tabel film, tiket, dan penonton, sistem dapat menampilkan judul film yang memiliki durasi paling pendek dan ditonton oleh penonton dengan umur paling muda. Pendekatan ini menunjukkan bagaimana subquery dapat digunakan untuk melakukan pencarian berdasarkan nilai ekstrem seperti minimum.

#### **5. Subquery untuk menampilkan teater yang paling sering digunakan**

Untuk mengetahui teater yang paling sering digunakan, pertama dilakukan proses penggabungan data antara tabel jadwal dan tiket menggunakan join. Setelah itu, dilakukan perhitungan jumlah penggunaan masing-masing teater menggunakan fungsi agregasi seperti COUNT. Hasil perhitungan ini kemudian dibandingkan dengan nilai maksimum yang diperoleh dari subquery yang menghitung total penggunaan setiap teater. Dengan demikian, query akan menampilkan nomor teater, kelas, dan total pemakaian dari teater yang paling sering digunakan. Penggunaan subquery dalam hal ini berfungsi untuk menentukan nilai maksimum sebagai pembanding dalam query utama.